

Binary Coded Decimal

BCD stands for Binary Coded Decimals. BCD is character code and not the number system. We are working in decimal number system but want to “write” each decimal digit in binary. We are not interested in representing the whole decimal number in binary. That is the reason they are also called as 8421 codes.

A decimal value **9287**₁₀ is to be expressed in BCD as

9					2					8					7			
8	4	2	1		8	4	2	1		8	4	2	1		8	4	2	1
1	0	0	1		0	0	1	0		1	0	0	0		0	1	1	1

It will make easier to conceptualize if we read it in reverse i.e.; DCB (Decimal coded in Binary). How many Decimals do we have as digits? The answer is, *ten digits* i.e.; from digit 0 to digit 9.

How many bits are required to represent a “decimal digit” in binary? The greatest among the ten digits is 9 and it will require only four bits to represent it in binary as shown in Table 1.

But, unfortunately four bit binary combination doesn’t stop at **1001**, it continues to generate **1010 (A)**, **1011 (B)**, **1100 (C)**, **1101 (D)**, **1110 (E)**, and **1111 (F)**. These six binary values are not valid to be included in our case. So we call them incorrect or invalid BCD codes.

Invalid BCD codes are A, B, C, D, E, and F.

Whenever we perform BCD arithmetic, we must keep in mind that the answer can be one of the *invalid BCD code*.

For example adding BCD 9 and BCD 1 will result in invalid BCD code A (i.e., **1010**).

We must correct this answer for which we have rules.

Table 1 Decimal Digits to Binary Representation

DECIMAL DIGITS	BINARY REPRESENTATION OF EACH DIGIT			
	B3 (MSB)	B2	B1	B0 (LSB)
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

Types of BCD Representation:

There are two types of BCD, *unpacked BCD* and *packed BCD*, which are described below.

Unpacked BCD

The unpacked BCD number is stored in one byte of memory location or a register of 8 bits, (see Appendix – A). It is stored using the lower 4 bits of a byte while the higher 4 bits remains zero. For example, the number **89** is represented as “**0000 1000 0000 1001**” in unpacked BCD.

Packed BCD

Two packed BCD numbers are stored in one byte of memory location or a register of 8 bits, (see Appendix – A). One number is stored in the lower 4 bits of a byte while the other number is stored in higher 4 bits. For example, the number **89** is represented as “**1000 1001**” in packed BCD. Due to this characteristic, packed BCD is considered twice as efficient in storing data.

Rules for correcting an invalid BCD arithmetic result:

Rule 1: If the *resulting digit is greater than BCD 9*, it is considered as **invalid**

Correction: to correct the result just add 6 (which is the number of invalid BCDs) to the result.

Example: 6 + 5

$$\begin{array}{r} 0110 \\ \underline{0101} \\ 1011 \end{array} \quad \text{with no auxiliary carry generated}$$

1011 is invalid BCD. For correction, add 6 to it

$$\begin{array}{r} 1011 \\ \underline{0110} \\ 0001 \underline{0001} \end{array} \quad \Rightarrow \text{correct, as } 6+5=11 \text{ in decimal}$$

Rule 2: If the *resulting digit generates a Carry called as Auxiliary Carry*, that digit is considered as **invalid**.

Correction: to correct the result just add 6 to the result.

Example: 8+8

$$\begin{array}{r} 1000 \\ \underline{1000} \\ 1 \underline{0000} \end{array} \quad \text{generating an auxiliary carry}$$

Result can be expressed as “0001 0000” $\Rightarrow$ 10 in decimal

This result is incorrect, though the digits are not greater than 9 but $8+8=16$ in decimal.

Since lower nibble (or lower digit) caused the auxiliary carry, therefore, we add 6 to the resulting lower digit.

$$\begin{array}{r} 0001 \underline{0000} \\ \underline{0110} \\ 0001 \underline{0110} \end{array} \quad \Rightarrow \text{correct as } 8+8=16 \text{ in decimal}$$

Auxiliary Carry:

A carry is generated by bit#3 (fourth bit starting from 0) to bit#4 (fifth bit) i.e., from one BCD digit to the next BCD digit.

Why adding 6 to correct the result: Since there are 6 invalid BCD codes and if we skip or ignore those 6 codes we will be counting correctly.

0 => 0000	10 => 0001 0000	20 => 0010 0000
1 => 0001	11 => 0001 0001	21 => 0010 0001
2 => 0010	12 => 0001 0010	22 => 0010 0010
• •	• • •	• • •
• •	• • •	• • •
• •	• • •	• • •
9 => 1001	19 => 0001 1001	29 => 0010 1001
1010	0001 1010	0001 1010
1011	0001 1011	0010 1011
1100	0001 1100	0010 1100
1101	0001 1101	0010 1101
1110	0001 1110	0010 1110
1111	0001 1111	0010 1111

APPENDIX – A**(i) Packed BCD**

If in one byte, two BCD digits are represented, then it will be termed as Packed BCD. That is, one BCD digit takes only 4-bits to get stored.

For example,

0 => 0000	10 => 0001 0000	20 => 0010 0000
1 => 0001	11 => 0001 0001	21 => 0010 0001
2 => 0010	12 => 0001 0010	22 => 0010 0010
• •	• • •	• • •
• •	• • •	• • •
• •	• • •	• • •
9 => 1001	19 => 0001 1001	29 => 0010 1001

(ii) Unpacked BCD

If in one byte, only one BCD digit is represented, then it will be termed as Unpacked BCD. That is, one BCD digit takes one byte to get stored.

For example,

0 => 00000000	10 => 00000001 00000000	20 => 00000010 00000000
1 => 00000001	11 => 00000001 00000001	21 => 00000010 00000001
2 => 00000010	12 => 00000001 00000010	22 => 00000010 00000010
• •	• • •	• • •
• •	• • •	• • •
• •	• • •	• • •
9 => 00001001	19 => 00000001 00001001	29 => 00000010 00001001